Instituto Tecnológico Superior Del oriente Del Estado De Hidalgo

Problemario

Docente:
Dr. Efrén Rolando Romero

1.- Error de Redondeo Binario

El error más común es intentar representar números decimales exactos (como `0.1`) en un sistema binario. Esto causa que operaciones simples devuelvan resultados inesperados.

- **Ejemplo:** `System.out.println(0.1 + 0.2);` imprime `0.30000000000000004` en lugar de `0.3`.
- **Causa:** Fracciones decimales no tienen una representación binaria finita.

Ejercicios:

1. ¿Cuál es el resultado de 0.1 + 0.2 en la mayoría de lenguajes?
2. Si sumas 0.1 tres veces, ¿obtendrás exactamente 0.3?
3. ¿Por qué 1.0 / 10.0 produce un error de redondeo pero 1.0 / 2.0 no?
4. En un sistema de 32 bits, ¿el error es mayor o menor que en uno de 64 bits?
5. ¿Cómo representarías 0.1 de forma exacta en binario?

Soluciones:

1. 0.30000000000000004.
2. No, suele ser ligeramente superior a 0.3.
3. Porque 2 es la base del sistema binario (2^{-1}), mientras que 10 no es una potencia de 2.
4. Mayor, porque tiene menos bits para la mantisa.
5. No se puede; es un número periódico infinito en binario: $0.0001100110011..._2

```java
public class RedondeoBinario {
    public static void main(String[] args) {
        double suma = 0.1 + 0.2;
        System.out.println("Resultado de 0.1 + 0.2: " + suma);
        // Salida: 0.30000000000000004

        // SOLUCIÓN: Usar BigDecimal para precisión exacta
        java.math.BigDecimal a = new java.math.BigDecimal("0.1");
        java.math.BigDecimal b = new java.math.BigDecimal("0.2");
        System.out.println("Solución con BigDecimal: " + a.add(b));
    }
}
```

2. Pérdida de Precisión por Magnitud(IEEE 754)

Cuando se operan números con magnitudes muy diferentes, el número más pequeño puede "desaparecer" debido a que no cabe en la mantisa del número más grande.

- **Ejemplo:** Sumar `1.0` a un `double` extremadamente grande puede no cambiar el valor original.

```java
public class PerdidaPrecision {
  public static void main(String[] args) {
    // Un double tiene aproximadamente 15-17 dígitos significativos de precisión
    double numeroGrande = 1.0e16; // 1 seguido de 16 ceros
    double numeroPequeno = 1.0;
```

```
        double resultado = numeroGrande + numeroPequeno;

        System.out.println("--- Demostración de Pérdida de Precisión ---");
        System.out.println("Número Grande:   " + numeroGrande);
        System.out.println("Número Pequeño:  " + numeroPequeno);
        System.out.println("Suma Resultante: " + resultado);

        // Verificación lógica
        if (resultado == numeroGrande) {
            System.out.println("\nRESULTADO: El número pequeño 'desapareció'.");
            System.out.println("La suma es igual al número original debido a la falta de bits en la mantisa.");
        }

        // --- SOLUCIÓN USANDO BIGDECIMAL ---
        java.math.BigDecimal bdGrande = new java.math.BigDecimal("1.0e16");
        java.math.BigDecimal bdPequeno = new java.math.BigDecimal("1.0");
        java.math.BigDecimal bdResultado = bdGrande.add(bdPequeno);

        System.out.println("\n--- Solución con BigDecimal ---");
        System.out.println("Suma Exacta:     " + bdResultado.toPlainString());
    }
}
```

Ejercicios:

1. Resultado de `100,000,000.0 + 0.0000001`.
2. ¿Qué pasa si a un contador `float` que llegó a 16,777,216 le sumas `1.0`?
3. ¿Por qué el orden de los sumandos afecta el resultado en grandes series?
4. ¿Cómo evitarías esto al sumar una lista de números muy variados?
5. Si `a = 1e16` y `b = 1`, ¿cuánto vale `(a + b) - a`?

Soluciones:

1. `100,000,000.0` (el pequeño se pierde).
2. Se queda en `16,777,216` (ya no tiene precisión para incrementos de 1).
3. Porque si sumas los pequeños primero, conservas su valor antes de llegar a la escala de los grandes.
4. Usando el **Algoritmo de Suma de Kahan**.
5. `0` (en lugar de `1`).

```java
public class PerdidaMagnitud {
    public static void main(String[] args) {
        float grande = 16777216f; // 2^24
        float pequeño = 1f;

        System.out.println("Grande: " + grande);
        System.out.println("Grande + Pequeño: " + (grande + pequeño));
        // Salida: 1.6777216E7 (¡Sigue siendo el mismo número!)

        // SOLUCIÓN: Usar double (mayor mantisa) o reorganizar la suma
    }
}
```

Comparación Directa con ==

Debido a los errores de redondeo mencionados, comparar dos números `double` usando == suele fallar.

- **Solución:** Se debe usar un margen de error (épsilon).

  java

  ```
  if (Math.abs(a - b) < 0.00001) { /* son iguales */ }
  ```

```
public class ComparacionDouble {
  public static void main(String[] args) {
    // 1. El Problema: Error de redondeo en aritmética de punto flotante
    double a = 0.1 + 0.1 + 0.1;
    double b = 0.3;

    System.out.println("Valor de a (0.1 * 3): " + a);
    System.out.println("Valor de b:          " + b);

    // 2. Intento de comparación directa (FALLARÁ)
    System.out.println("\n--- Comparación con '==' ---");
    if (a == b) {
      System.out.println("Resultado: Son iguales");
    } else {
      System.out.println("Resultado: SON DIFERENTES (Error esperado)");
    }

    // 3. La Solución: Uso de un margen de error (Épsilon)
    double epsilon = 0.00001; // Definimos la tolerancia

    System.out.println("\n--- Comparación con Épsilon (" + epsilon + ") ---");
    if (Math.abs(a - b) < epsilon) {
      System.out.println("Resultado: Son iguales (dentro del margen de error)");
    } else {
      System.out.println("Resultado: Son diferentes");
    }
  }
}
```

Ejercicios:

1. ¿Es `(0.1 + 0.1 + 0.1) == 0.3` verdadero o falso?
2. ¿Por qué nunca debes usar == con `double` en un `if`?
3. ¿Cuál es la forma correcta de comparar `a` y `b` flotantes?
4. ¿Qué es el valor "Epsilon"?
5. ¿Qué pasa si comparas `0.0 == -0.0`?

Soluciones:

1. Falso.
2. Porque el redondeo interno hace que los bits finales no coincidan.
3. `abs(a - b) < epsilon`.
4. El error máximo esperado o la diferencia más pequeña representable.
5. Suele dar `true` (por estándar IEEE 754), aunque sus bits sean diferentes.

```java
public class ComparacionFlotante {
    public static void main(String[] args) {
        double a = 0.1 + 0.1 + 0.1;
        double b = 0.3;

        System.out.println("¿Es 0.1 * 3 == 0.3? " + (a == b)); // false

        // SOLUCIÓN: Comparar con una tolerancia (Epsilon)
        double epsilon = 0.000001;
        boolean sonIguales = Math.abs(a - b) < epsilon;
        System.out.println("¿Son iguales con tolerancia? " + sonIguales); // true
    }
}
```

4. Acumulación de Errores en Bucles

Realizar miles de operaciones aritméticas consecutivas con `double` acumula pequeños errores de redondeo que pueden resultar en una desviación significativa al final del proceso.

```java
import java.math.BigDecimal;



public class AcumulacionErroresBucle {
   public static void main(String[] args) {
      int iteraciones = 1000000; // Un millón de sumas
      double incremento = 0.1;

      // 1. Acumulación usando 'double'
      double sumaDouble = 0.0;
      for (int i = 0; i < iteraciones; i++) {
         sumaDouble += incremento;
      }

      // El resultado esperado debería ser exactamente 100,000.0
      double esperado = iteraciones * incremento;

      System.out.println("--- Acumulación en Bucle (1,000,000 de iteraciones) ---");
      System.out.println("Resultado esperado: " + esperado);
      System.out.println("Resultado double:   " + sumaDouble);
      System.out.println("Diferencia (Error): " + (sumaDouble - esperado));

      // 2. Solución usando 'BigDecimal' para precisión absoluta
      BigDecimal sumaBD = BigDecimal.ZERO;
      BigDecimal incrementoBD = new BigDecimal("0.1");

      for (int i = 0; i < iteraciones; i++) {
         sumaBD = sumaBD.add(incrementoBD);
      }

      System.out.println("\n--- Solución con BigDecimal ---");
      System.out.println("Resultado exacto:   " + sumaBD.toPlainString());

      // 3. Verificación de error
```

```
    if (sumaDouble != esperado) {
        System.out.println("\nNOTA: El error de 'double' es notable tras un millón de sumas.");
    }
  }
}
```

Ejercicios:

1. Si sumas 0.0001 en un bucle 10,000 veces para llegar a 1.0, ¿qué obtendrás?
2. ¿Qué es mejor para un bucle: for(float i=0; i<1; i+=0.1) o for(int i=0; i<10; i++)?
3. En simulaciones físicas, ¿por qué el tiempo (t) suele fallar tras muchas iteraciones?
4. ¿Cómo afecta esto al cálculo de intereses compuestos en finanzas?
5. ¿Qué tipo de dato usarías para evitar esto en dinero?

Soluciones:

1. Un número cercano a 0.999... o 1.000..., rara vez 1.0 exacto.
2. El de int (siempre usa enteros para control de bucles).
3. Porque sumas pequeños diferenciales de tiempo dt que acumulan error.
4. El error se multiplica, resultando en pérdidas o ganancias ficticias.
5. Decimal (en C# o Python) o enteros (centavos).

```java
public class AcumulacionBucle {
    public static void main(String[] args) {
        double valor = 0.0;
        for (int i = 0; i < 100; i++) {
            valor += 0.1;
        }
        System.out.println("Suma de 0.1 cien veces: " + valor);
        // Salida: 9.99999999999998 (No es 10.0)

        // SOLUCIÓN: Usar un contador entero para el bucle
        // for (int i = 0; i < 100; i++) { double real = i / 10.0; }
    }
}
```

Cancelación por Resta (Loss of Significance)

Ocurre cuando restas dos números muy cercanos entre sí. La mayoría de los dígitos significativos se cancelan, dejando solo el error de redondeo como el "resultado" aparente.

java

```java
public class CancelacionPorResta {
    public static void main(String[] args) {
        // Dos números muy grandes y muy cercanos
        double x = 1234567890.1234567;
        double y = 1234567890.1234560;
```

```
        // El resultado esperado es 0.0000007
        double resultado = x - y;

        System.out.println("Resultado real: " + resultado);
        // En 2026 notarás que el resultado es 0.0 o un valor basura
        // debido a que los últimos dígitos se perdieron al almacenar x e y.
    }
}
```

Ejercicios:

1. Si x = 1.23456789y y = 1.23456780, ¿cuántos dígitos significativos quedan al restar?
2. ¿Por qué es peligroso calcular la fórmula cuadrática -b + sqrt{b^2 - 4ac} cuando 4ac es muy pequeño?
3. Evalúa 1 - \cos(x) para x muy pequeño. ¿Qué problema hay?
4. ¿Cómo reescribirías una resta para evitar la cancelación?
5. ¿Qué sucede con la precisión relativa en este proceso?

Soluciones:

1. Solo uno (0.00000009$), el resto es "ruido".
2. Porque \sqrt{b^2} será casi igual a b, y al restarlos perdemos precisión.
3. cos(x) será casi 1.0, resultando en una cancelación catastrófica.
4. Usando identidades trigonométricas o racionalización.
5. La precisión relativa cae drásticamente (el error relativo aumenta).

```
public class CancelacionResta {
    public static void main(String[] args) {
        double x = 1.0000000000001;
        double y = 1.0;
        double resultado = x - y;

        System.out.println("Resultado de la resta: " + resultado);
        // Salida: 9.992007221626409E-14 (Debería ser 1E-13)

        // SOLUCIÓN: Reestructurar la fórmula matemática para evitar restas cercanas.
    }
}
```

Desbordamiento Silencioso (Overflow)

A diferencia de los errores de decimales, este ocurre con tipos enteros (`int`, `long`). Si superas el valor máximo, Java no lanza una excepción; simplemente "da la vuelta" al número más pequeño (negativo), lo que destruye la exactitud del cálculo.

java

```
public class DesbordamientoEntero {
    public static void main(String[] args) {
        int max = Integer.MAX_VALUE; // 2,147,483,647
        int resultado = max + 1;
```

```
        System.out.println("Máximo int: " + max);
        System.out.println("Máximo + 1: " + resultado); // Imprime -2147483648

        // Solución: Usar Math.addExact para lanzar una excepción si ocurre
        try {
            Math.addExact(max, 1);
        } catch (ArithmeticException e) {
            System.out.println("Error detectado: ¡Desbordamiento!");
        }
    }
}
```

Ejercicios:

1. En un `signed char` (máximo 127), ¿qué resulta de `127 + 1`?
2. ¿Por qué `abs(MIN_INT)` a veces devuelve un número negativo?
3. Si `a` y `b` son positivos, ¿cómo puede `a + b` ser menor que `0`?
4. ¿Qué sucede en Python con los enteros vs C++?
5. ¿Cómo detectarías un desbordamiento antes de que ocurra?

Soluciones:

1. `-128`.
2. Porque el rango negativo suele tener un elemento más que el positivo.
3. Por desbordamiento de enteros con signo.
4. Python gestiona memoria dinámica para enteros (no desbordan); C++ tiene tamaño fijo.
5. Comprobando: `if (a > MAX_INT - b)`.

```
public class Desbordamiento {
    public static void main(String[] args) {
        int max = Integer.MAX_VALUE; // 2,147,483,647
        int resultado = max + 1;

        System.out.println("Máximo + 1: " + resultado);
        // Salida: -2147483648 (¡Se volvió negativo!)

        // SOLUCIÓN: Usar Math.addExact() para que lance una excepción si ocurre.
        // int seguro = Math.addExact(max, 1); // Lanza ArithmeticException
    }
}
```

Conversión Estrecha (Narrowing Primitive Conversion)

Al convertir un tipo de mayor capacidad a uno menor (de `double` a `int` o de `long` a `int`), Java simplemente trunca los bits sobrantes, lo que puede cambiar drásticamente el valor original sin previo aviso.

Java

```java
public class ErrorConversion {
    public static void main(String[] args) {
        double valorDouble = 3.14159e10; // Un número muy grande
        int valorInt = (int) valorDouble; // Casting explícito

        System.out.println("Valor Original: " + valorDouble);
        System.out.println("Valor Truncado: " + valorInt);
        // Imprime 2147483647 (el máximo int) porque el double no cabe.
    }
}
```

Ejercicios:

1. Si conviertes `3.99` a un `int`, ¿cuál es el resultado?
2. ¿Qué pasa al meter un `long` de 64 bits en un `int` de 32?
3. ¿Por qué el compilador suele dar un "Warning" en estos casos?
4. ¿Cuál es la diferencia entre truncar y redondear?
5. ¿Qué valor toma `(int)1e20`?

Soluciones:

1. `3` (se eliminan los decimales, no se redondea).
2. Se pierden los 32 bits superiores (pérdida de magnitud masiva).
3. Porque es una operación "insegura" que causa pérdida de información.
4. Truncar es cortar hacia el cero; redondear busca el entero más cercano.
5. Depende del lenguaje, pero suele ser el valor máximo del `int` o basura.

```java
public class Narrowing {
    public static void main(String[] args) {
        double decimal = 9.99;
        int entero = (int) decimal; // Casting explícito

        System.out.println("Double 9.99 convertido a int: " + entero);
        // Salida: 9 (Se pierde el .99, no se redondea)

        long muyGrande = 3000000000L;
        int pequeño = (int) muyGrande;
        System.out.println("Long 3 mil millones a int: " + pequeño);
        // Salida: -1294967296 (Basura por pérdida de bits)

        // SOLUCIÓN: Verificar rangos antes de convertir o usar Math.round().
    }
}
```